마이크로프로세서응용실습 프로젝트

13조-2021440073_안한영, 2023440095_이은지

I. Introduction

A. 프로젝트 주제

본 프로젝트는 제스처 기반 텍스트·음성 변환 시스템 설계로, 플렉스 센서 기반 손동작 인식 시스템을 이용하여 수어 동작을 한글 문자로 변환하는 스마트 입력 장치를 개발한다.

B. 프로젝트 동작 목표

플렉스 센서를 통해 사용자의 수화 동작을 데이터화하고, 이를 한글 자모로 변환하여 완성된 문장으로 출력할 수 있는지 검증한다.

기본 설정 및 맵핑 로직

엄지손가락: 모드 스위치 (High=자음 모드, Low=모음 모드).

나머지 네 손가락: 굽힘/폄 상태 조합(2^4=16 가지 패턴)에 따라 사전 정의된 자모(CHO_MAP, JUNG_MAP) 매핑

Threshold: ADC 값 2000 (약 1.6V) 기준 이진화 (Low=1/굽힘, High=0/폄)

Debouncing: 400ms (0.4 초) 이상 동작 유지 시 유효 입력으로 인정

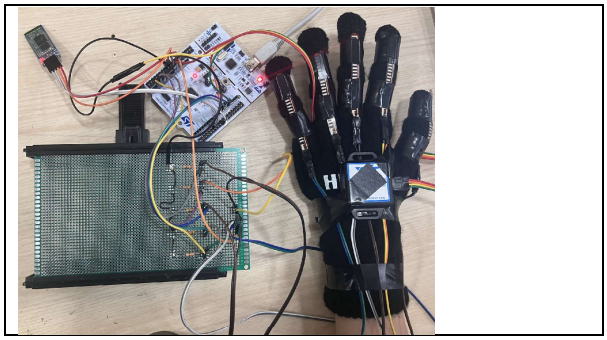
시스템 구조

1. flex sensor 값을 5개의 ADC 채널을 통해 읽어 자음 모음 결정
2. 다섯 손가락을 전부 구부리면 자음 모음을 조합해 UTF-8로 변환하여 한글 글자 생성
3. imu 센서를 통해 손바닥을 180도 회전하면 생성한 글자를 출력하도록 함
4. 지정해둔 글자가 입력되었을 때, 글자 자동 완성하여 출력

II. SYSTEM DESIGN

A. Hardware Design

1. MCU: STM32F103RB
   - 시스템의 메인 컨트롤러로, 모든 센서 데이터 수집 및 처리, PC와의 통신을 담당
   - 시스템 클록: 72MHz
   - 전원: USB(5V)를 통해 공급받으며, 내부 3.3V 레귤레이터를 통해 센서 및 로직 전원을 공급
2. 센서
   - 플렉스 센서 x 5
     - 손가락마다 플렉스 센서를 연결해 손가락의 굽힘 정도를 측정
     - 회로 구성: 3.3V 풀 다운 방식 전압 분배 회로 적용
     - 저항 값: 33kΩ 고정 저항 사용 (센서 범위 25kΩ ~100kΩ).
     - 신호: 굽힐수록 저항 증가 -> ADC 입력 전압 감소
   - IMU 센서 (WT901C)
     - 손목의 회전(Roll) 및 움직임을 감지
     - 통신: UART
3. 통신 및 출력
   - PC 연결: HC-06 블루투스 모듈을 활용한 무선 통신 및 TTS 음성 출력
   - TTS (Text To Speech): PC 파이썬 라이브러리 (pyttsx3)를 활용하여 수신된 텍스트를 음성으로 출력
4. 하드웨어 사진



B. Software Design

0. IOC 설정

- ADC & DMA
  - 기능: 5개의 플렉스 센서 값을 CPU 개입 없이 실시간으로 수집하기 위해 ADC와 DMA를 사용
  - 설정: 12비트 해상도 (0~4095), 연속 변환 모드

- 이유: 센서 값을 읽느라 CPU가 멈추지 않게 하여, 동시에 UART 통신이나 로직 처리를 원활하게 하기 위함

・UART

- USART1: IMU 센서 데이터 수신
- USART2: PC와의 명령어 수신 (입력된 자음 모음과 조합된 글자 확인을 위한 출력)
- USART3: HC-06 블루투스 통신 (출력할 문장을 띄움)

1. 플렉스 센서 값 읽기 & 자음 모음 결정

Flex sensor의 값이 ADC로 들어오고 이를 DMA로 받아 adc_values[]에 저장한다.

```
int get_finger_pattern(void)
{
    int pattern = 0;
    if (adc_values[1] < THRESHOLD) pattern |= 1; //0001
    if (adc_values[2] < THRESHOLD) pattern |= 2; //0010
    if (adc_values[3] < THRESHOLD * 0.9) pattern |= 4; //0100
    if (adc_values[4] < THRESHOLD) pattern |= 8; //1000
    return pattern;
}
```

get_finger_pattern() 함수로 들어온 adc 값에 따라 패턴을 결정한다. 각 손가락마다 문턱전압 값이 2000, 즉 1.611V 이하로 내려가면 = 굽힘으로 인식한다. 다섯 손가락 굽힘 유무를 조합하여 패턴 값을 결정한다.

```
const int CHO_MAP[16] = {-1,0,2,3,5,6,7,9,11,12,14,15,16,17,18,-1};
const int JUNG_MAP[16] = {-1,0,2,4,6,8,12,13,17,18,20,1,5,-1,-1,-1};
```

결정한 패턴 값은 process_hand_gesture() 함수에 넣어 자음 모음 타입을 결정하고, 각 인덱스로 어떤 자음 모음인지 결정할 것이다.

이 때 자음 모음 입력을 유니코드 인덱스로 바꿔주기 위한 매핑 테이블 CHO_MAP과 JUNG_MAP을 선언한다.

```
void process_hand_gesture(void)
{
    update_robot_hand_linear();

    if (adc_values[0] < THRESHOLD &&
        adc_values[1] < THRESHOLD &&
        adc_values[2] < THRESHOLD &&
        adc_values[3] < THRESHOLD &&
        adc_values[4] < THRESHOLD)
    {
        send_flag = 1;
        return;
    }
```

전부 다 구부렸을 때는 send flag를 1로 만들어 글자를 전송하도록 한다.

```
int curr_pattern = get_finger_pattern();
int curr_mode = (adc_values[0] < THRESHOLD);
```

0번(엄지) 값으로 mode를 결정한다. mode가 0일 때는 자음, 1일 때는 모음이다.

```
if (curr_pattern == last_pattern && curr_mode == last_mode)
{
    if (is_registered == 0)
    {
        if (HAL_GetTick() - stable_start_tick >= STABLE_TIME)
        {
            if (input_idx < MAX_INPUT)
            {
                int mapped_idx = -1;
                InputType type = TYPE_NONE;
                if (curr_mode == 0)
                {
                    mapped_idx = CHO_MAP[curr_pattern];
                    type = TYPE_CHO;
                }
                else
                {
                    mapped_idx = JUNG_MAP[curr_pattern];
                    type = TYPE_JUNG;
                }

                if (mapped_idx != -1)
                {
                    input_buffer[input_idx].type = type;
                    input_buffer[input_idx].index = mapped_idx;
                    input_idx++;
```

STABLE_TIME은 0.4초로, 같은 입력이 0.4초동안 지속될 때 입력으로 간주한다. mode가 0인 경우, type을 초성으로 결정하고, 1인 경우 중성으로 간주한다.

```
if (type == TYPE_CHO)
{
    HAL_UART_Transmit(&huart2, CHO_UTF8[mapped_idx], 3, 100);
}
else if (type == TYPE_JUNG)
{
    HAL_UART_Transmit(&huart2, JUNG_UTF8[mapped_idx], 3, 100);
}
```

바로바로 들어온 자음 모음을 확인하기 위해. 자음 모음 경우에 따라 매핑된 글자를 huart2로 전송한다.

2. 자음 모음 조합 후 글자 만들기

손가락이 다 굽힌 경우 send_flag값이 1이 되고, 이 경우 assemble_and_send() 함수를 호출하도록 한다.

```
void assemble_and_send(void)
{
    int i = 0;
    uint8_t utf8_char[5];

    while (i < input_idx)
    {
        if (input_buffer[i].type == TYPE_CHO &&
            (i+1 < input_idx) &&
            input_buffer[i+1].type == TYPE_JUNG)
        {
            int cho = input_buffer[i].index;
            int jung = input_buffer[i+1].index;
            int jong = 0;
            int consumed = 2;
```

input_buffer 안에 들어있는 자음 모음을 차례대로 분석한다. 초성+종성 조합일 때. 즉 받침이 없을 때와 받침이 있을때로 나누어 처리한다.

받침이 없는 경우, 초성 인덱스를 cho에 저장하고 중성 인덱스를 jung에 저장하고, 종성은 없기 때문에 0이다.

```
if ((i+2 < input_idx) && input_buffer[i+2].type == TYPE_CHO)
{
    int t = cho_to_jong(input_buffer[i+2].index);
    if (t != 0)
    {
        jong = t;
        consumed = 3;
    }
}
```

받침이 있는 경우. cho_to_jong() 함수를 호출해 jong 값을 결정한다.

```
int cho_to_jong(int cho_idx)
{
    switch(cho_idx) {
        case 0: return 1;
        case 2: return 4;
        case 3: return 7;
        case 5: return 8;
        case 6: return 16;
        case 7: return 17;
        case 9: return 19;
        case 11: return 21;
        case 12: return 22;
        case 14: return 23;
        case 15: return 24;
        case 16: return 25;
        case 17: return 26;
        case 18: return 27;
        default: return 0;
    }
}
```

cho_to_jong() 함수는 초성을 종성으로 변환하기 위한 매핑 테이블이다. 초성에 해당하는 자음의 인덱스를 종성에 해당하는 인덱스로 변환한다.

```
make_hangul_utf8(cho, jung, jong, utf8_char);
HAL UART Transmit(&huart2, utf8 char, 3, 100);
```

이제 make_hangul 함수를 호출해 cho, jung, jong 변수를 넣어 하나의 글자로 조합한 utf8 char 값을 반환받고, 이를 huart2로 송신해 확인할 수 있도록 한다.

```
void make_hangul_utf8(int cho, int jung, int jong, uint8_t* buf)
{
    uint16_t unicode = 0xAC00 + (cho * 588) + (jung * 28) + jong;

    buf[0] = 0xE0 | ((unicode >> 12) & 0x0F);
    buf[1] = 0x80 | ((unicode >> 6) & 0x3F);
    buf[2] = 0x80 | (unicode & 0x3F);
    buf[3] = '\0';
}
```

make_hangul 함수는 cho, jung, hong 값을 유니코드 변환 공식에 넣어 유니코드 값을 반환한다.

```
        if (sentence_idx + 3 < 100)
        {
          memcpy(&sentence_buffer[sentence_idx], utf8_char, 3);
          sentence_idx += 3;
        }

        i += consumed;
    }
    else
    {
        i++;
    }


AL_UART_Transmit(&huart2, (uint8_t*)"\r\n", 2, 100);
```

변환한 값들은 sentence_buffer에 차례로 저장하고, 글자들이 모인 문장인 sentence _buffer는 이후에 send_flag가 2가 되면 출력하도록 할 것이다.

3. imu 센서

imu 센서로 손바닥 뒤집기를 감지해 글자 출력할 것이다. 이를 위해 imu 센서 값을 파싱하는 코드를 작성한다.

```
void parse_imu_packets(void)
{
    uint8_t header[2];
    uint8_t data_bytes[8];
    uint8_t sum;
    uint8_t check_sum = 0;

    while (RingBuffer_GetCount(&imu_ring_buffer) >= WT_PACKET_SIZE)
    {
        header[0] = RingBuffer_Peek(&imu_ring_buffer, 0);
        header[1] = RingBuffer_Peek(&imu_ring_buffer, 1);

        if (header[0] != 0x55)
        {
            RingBuffer_Read(&imu_ring_buffer);
            continue;
        }

        RingBuffer_Read(&imu_ring_buffer);
        RingBuffer_Read(&imu_ring_buffer);

        check_sum = header[0] + header[1];

        for (int i = 0; i < 8; i++)
        {
            if (RingBuffer_IsEmpty(&imu_ring_buffer)) return;
            data_bytes[i] = RingBuffer_Read(&imu_ring_buffer);
            check_sum += data_bytes[i];
        }

        if (RingBuffer_IsEmpty(&imu_ring_buffer)) return;
        sum = RingBuffer_Read(&imu_ring_buffer);

        if (check_sum != sum)
            continue;
```

패킷 헤더(0x55)가 나타날 때까지 링버퍼를 읽는다. RingBuffer_GetCount() 함수는 콜백함수에서 입력 받은 &imu_ring_buffer가 WT_PACKET_SIZE, 11바이트일 경우 실행하고, &imu_ring_buffer 값 읽어서 전부 data_bytes[] 배열에 넣는다.

```
        int16_t x_raw = (data_bytes[1] << 8) | data_bytes[0];
        int16_t y_raw = (data_bytes[3] << 8) | data_bytes[2];
        int16_t z_raw = (data_bytes[5] << 8) | data_bytes[4];
```

DATAL, DATAH를 결합해서 16bit raw Data 만든다.

```c
switch (header[1])
{
    case ANGLE_DATA_ID: //0x53
        imu_data.roll  = x_raw / 32768.0f * 180.0f;
        imu_data.pitch = y_raw / 32768.0f * 180.0f;
        imu_data.yaw   = z_raw / 32768.0f * 180.0f;
        data_updated_flag |= 0x01;
        break;

    case ACCEL_DATA_ID: //0x51
        imu_data.acc_x = x_raw / 32768.0f * 16.0f;
        imu_data.acc_y = y_raw / 32768.0f * 16.0f;
        imu_data.acc_z = z_raw / 32768.0f * 16.0f;
        data_updated_flag |= 0x02;
        break;

    case GYRO_DATA_ID: //0x52
        imu_data.gyro_x = x_raw / 32768.0f * 2000.0f;
        imu_data.gyro_y = y_raw / 32768.0f * 2000.0f;
        imu_data.gyro_z = z_raw / 32768.0f * 2000.0f;
        data_updated_flag |= 0x04;
        break;

    case MAG_DATA_ID: //0x54
        imu_data.mag_x = x_raw / 32768.0f * 4912.0f;
        imu_data.mag_y = y_raw / 32768.0f * 4912.0f;
        imu_data.mag_z = z_raw / 32768.0f * 4912.0f;
        data_updated_flag |= 0x08;
        break;
}
```

TYPE 값에 따라 데이터를 계산하도록 한다.

```c
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    // IMU UART1
    if (huart->Instance == USART1)
    {
        RingBuffer_Write(&imu_ring_buffer, wt901c_rx_buffer[0]);
        HAL_UART_Receive_IT(&huart1, wt901c_rx_buffer, 1);
    }
}
```

이렇게 계산한 imu 값은 interrupt 방식으로 huart 1에서 imu 센서 값 수신 받고, 수신 시 링버퍼에 센서 값을 입력한다.

4. main 함수 동작

```c
while (1)
{
  // FLEX 센서
  process_hand_gesture();
  if (send_flag == 1)
  {
      send_flag = 0;
      assemble_and_send();
      input_idx = 0;
      memset(input_buffer, 0, sizeof(input_buffer));

      assemble_roll=imu_data.roll;
      send_count =0;
  }
```

send_flag 가 1일 때, 글자를 조합하기 위해 assemble_and_send() 함수를 호출한다.

또한 글자가 조합된 시점의 roll 값을 저장해 이후 손바닥이 180도 회전한 것을 감지할 수 있도록 한다.

```c
else if (send_flag == 2)
{
    HAL_UART_Transmit(&huart2, (uint8_t*)"2\n", 2, 100);

    send_flag = 0;
    send_count =0;


    int eff_len = sentence_idx;

    while (eff_len > 0 &&
          (sentence_buffer[eff_len - 1] == '\r' ||
           sentence_buffer[eff_len - 1] == '\n' ||
           sentence_buffer[eff_len - 1] == ' '))
    {
        eff_len--;
    }
```

send flag가 2일 때, 출력하도록 한다. 출력은 자동완성 맵핑 글자일 경우와 아닐 경우로 나누어 동작하도록 한다.

```c
int replaced = 0;
for (int i = 0; i < mapping_count; i++)
{

    if (eff_len == table[i].key_len &&
        memcmp(sentence_buffer, table[i].key, table[i].key_len) == 0) //키일
    {
        HAL_UART_Transmit(&huart2, table[i].value, table[i].value_len, 100);
        HAL_UART_Transmit(&huart3, table[i].value, table[i].value_len, 100);

        HAL_UART_Transmit(&huart2, (uint8_t*)"\r\n", 2, 100);
        replaced = 1;
        break;
    }
}
```

sentence buffer에 저장된 문자가 자동완성 지정 키일 경우, table[]에서 맵핑된 결과 값을 sentence_buffer에 저장하도록 한다.

sentence_buffer 값은 huart 2로 전송해 파이썬에서 tts 라이브러리를 통해 소리를 읽고, huart 3로 전송해 블루투스로 글자를 출력한다.

```c
MappingEntry table[] = {
    { key_annyeong, sizeof(key_annyeong), val_annyeonghaseyo, sizeof(val_annyeonghaseyo) },
    { key_i,        sizeof(key_i),        val_i,              sizeof(val_i)              },
    { key_an,       sizeof(key_an),       val_an,             sizeof(val_an)             },
    { key_ip,       sizeof(key_ip),       val_ip,             sizeof(val_ip)             },
    { key_iseung,   sizeof(key_iseung),   val_iseung,         sizeof(val_iseung)         },
    { key_gamsa,    sizeof(key_gamsa),    val_gamsa,          sizeof(val_gamsa)          }
};
int mapping_count = sizeof(table) / sizeof(table[0]);
```

table[] 은 '안녕', '이', '안', '입', '이승', '감사' 가 입력될 경우, 각각 '안녕하세요', '이은지', '안한영', '입니다', '이승환교수님', '감사합니다'로 출력하도록 맵핑되어 있다. 각 키들은 해당 utf-8 바이트로 지정되어있다.

```c
if (!replaced)
{
    HAL_UART_Transmit(&huart2, sentence_buffer, eff_len, 100);
    HAL_UART_Transmit(&huart3, sentence_buffer, eff_len, 100);

    HAL_UART_Transmit(&huart2, (uint8_t*)"\r\n", 2, 100);

    HAL_Delay(100);
}
```

지정된 키가 아니라 일반 문자가 입력되었을 경우에는 바로 sentence_buffer를 huart2와 huart3에 송신해 출력하도록 한다.

```
parse_imu_packets();

HAL_Delay(5);


curr= imu_data.roll;
base = assemble_roll;

d=curr-base;

if(fabs(d)>=180.0f){
    send_count=1;
}

if(send_count) {
    send_flag=2;
}
```

imut 파싱 함수를 호출한다. 현재 roll 값을 curr에 저장하고, send flag가 1일 때의 roll 값을 base에 저장하고, 이 차이가 180 이상일시 send flag를 2로 바꾸도록 한다.

5. 음성 출력

파이썬의 tts 기능을 이용해 글자를 읽는다.

```
PORT = 'COM3'
BAUDRATE = 115200

try:
    ser = serial.Serial(PORT, BAUDRATE, timeout=1)
    print(f"{PORT} connected (baud {BAUDRATE})")
except Exception as e:
    print(f"Serial connect failed: {e}")
    sys.exit()
```

serial port와 baudrate를 설정한다.

```
def speak_once(text):
    """Speak with a fresh TTS engine each time (stable)."""
    try:
        engine = pyttsx3.init()
        engine.setProperty('rate', 150)
        engine.setProperty('volume', 1.0)
        engine.say(text)
        engine.runAndWait()
        engine.stop()
    except Exception as e:
        print(f"TTS error: {e}")
```

pyttsx3 TTS 엔진을 생성하여 전달받은 text를 한 번 말하게 한다.

```
long map(long x, long in_min, long in_max, long out_min, long out_max) {
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}

//범위 제한
long constrain(long x, long a, long b) {
    if (x < a) return a;
    if (x > b) return b;
    return x;
}

// 로봇 손 제어
void update_robot_hand_linear(void)
{
    for (int i = 0; i < 5; i++)
    {
        long adc_val = adc_values[i];

        long pulse = map(adc_val, ADC_FLAT, ADC_BENT, SERVO_MIN, SERVO_MAX);
        pulse = constrain(pulse, SERVO_MIN, SERVO_MAX);

        switch (i)
        {
            case 0: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, pulse); break;
            case 1: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, pulse); break;
            case 2: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, pulse); break;
            case 3: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, pulse); break;
            case 4: __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse); break;
        }
    }
}
```

```
tts_trigger = False   # '2' 받은 뒤 True → 다음 줄을 TTS

def listen_thread():
    global tts_trigger
    print("Listening for incoming text...")

    while True:
        try:
            raw = ser.readline().decode('utf-8', errors='ignore').strip()

            if not raw:
                continue

            if raw == "2":
                tts_trigger = True
                continue
            if tts_trigger:

                code = ord(raw[0])
                if 0xAC00 <= code <= 0xD7A3:
                    speak_once(raw)

                tts_trigger = False

            print(f"[RX] {raw}")
        except Exception as e:
            print(f"Unexpected error: {e}")
            break
```

UART에서 문자를 받아, '2'가 들어오면 다음 한글 문자를 TTS로 읽어준다.

6. 서보 모터 제어 (삭제)

```
long map(long x, long in_min, long in_max, long out_min, long out_max) {
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}

//범위 제한
long constrain(long x, long a, long b) {
    if (x < a) return a;
    if (x > b) return b;
    return x;
}

// 로봇 손 제어
void update_robot_hand_linear(void)
{
    for (int i = 0; i < 5; i++)
    {
        long adc_val = adc_values[i];

        long pulse = map(adc_val, ADC_FLAT, ADC_BENT, SERVO_MIN, SERVO_MAX);
        pulse = constrain(pulse, SERVO_MIN, SERVO_MAX);

        switch (i)
        {
            case 0: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, pulse); break;
            case 1: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, pulse); break;
            case 2: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, pulse); break;
            case 3: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, pulse); break;
            case 4: __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse); break;
        }
    }
}
```

III. EXPERIMENTAL RESULTS

A. Experiment #1- Hand Gesture Recognition & Text Translation

· 목표

- 동작 인식: 사용자가 'ㅇ', 'ㅏ', 'ㄴ' 동작을 각각 0.4 초간 유지했을 때, STM32 가 이를 순차적으로 인식하여 내부 버퍼에 저장함
- 한글 조합: '1'번 명령(조립) 수신 시, [ㅇ, ㅏ, ㄴ] 데이터가 오토마타 알고리즘을 통해 "안"이라는 완성형 한글로 정확히 변환됨
- 데이터 전송: 손을 180 도 회전하여 IMU 센서가 180 도 감지 시 '2'번 명령(전송) 시, 완성된 문장이 UART 통신을 통해 PC 및 블루투스 모듈로 전송되었으며, 파이썬 스크립트가 이를 수신하여 TTS 로 음성을 출력함
- 자동 완성: "안녕", "이승"과 같은 특정 패턴 입력 시, "안녕하세요", "이승환교수님" 등으로 자동 변환되어 전송됨을 확인.

· 결과

- cube ide live expression

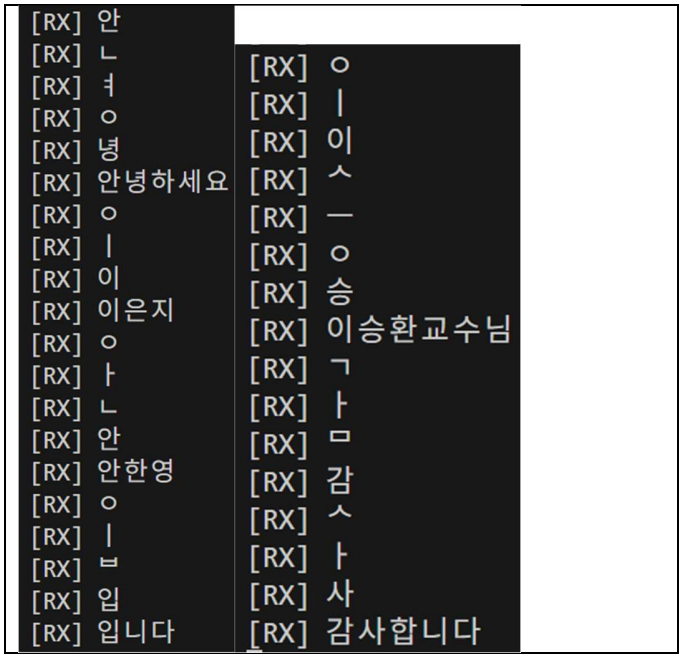
adc_values	volatile uint16_t [5]	[5]
adc_values[0]	volatile uint16_t	2131
adc_values[1]	volatile uint16_t	2267
adc_values[2]	volatile uint16_t	2137
adc_values[3]	volatile uint16_t	2308
adc_values[4]	volatile uint16_t	2118

cube ide 의 live expression 을 보면, adc value 5 개가 올바르게 들어오는 것을 확인할 수 있다. 현재 다섯 손가락 모두 펴져있는 상태로, 굽힐 시 2000 이하로 내려갈 것이다.

imu_data.roll	float	-24.6038818
assemble_roll	volatile float	14.7161865

imu 센서의 roll 값도 잘 들어오고 있는 것을 확인할 수 있다. imu_data.roll 값은 지금 읽고 있는 현재 imu 의 roll 값이고, assemble_roll 은 send_flag 가 1 일 때, 즉 모든 손가락을 구부렸을 때 저장될 roll 값으로, 이 둘의 차이가 180 이상일 때 send_flag 값이 2 가 되며 글자를 전송할 것이다.

- huart2: 파이썬 터미널

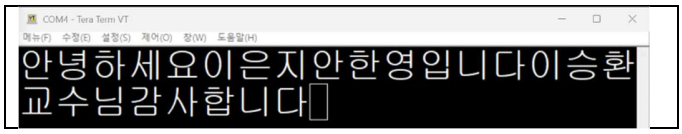


파이썬 터미널을 보면, 입력되는 자음 모음이 들어올 때 바로바로 잘 출력되고 있다. (ㄴ, ㅕ, ㅇ)

send_flag 가 1 이 되어 글자를 조합할 때도 글자가 잘 조합되어 출력되고 있음을 확인할 수 있다. (안, 녕)

또한 send_flag 가 2 가 되어 완성된 문장을 출력하는 것도 확인할 수 있으며, 자동완성 키가 입력되었을 때 맵핑된 결과가 출력되는 것도 확인할 수 있다. (안녕하세요)

- huart3: 블루투스 통신



블루투스로 연결한 tera term 화면을 보면, 출력 결과만을 올바르게 출력하고 있다.

B. Experiment #2-Robotic Hand Control

서보모터(GG90)를 이용해 로봇 손가락을 구동하도록 하였다.

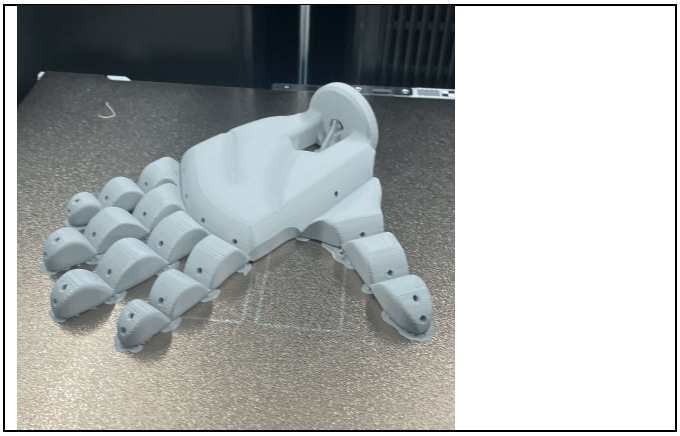
- 제어 방식: 50Hz PWM (20ms 주기), 펄스 폭 1.0ms~2.0ms 제어
- IOC 설정: PWM 타이머 이용

  TIM1 (CH1~CH4): 엄지, 검지, 중지, 약지 제어.

  TIM2 (CH1): 새끼손가락 제어.

  주파수: 72MHz 클럭을 분주하여 1MHz (1μs 단위) 카운터 생성, 주기 20ms (50Hz) 설정.

- 하드웨어 구성



3D 프린터를 이용해 로봇손을 출력하였다.

- 소프트웨어 구성

```
long map(long x, long in_min, long in_max, long out_min, long out_max) {
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}

//범위 제한
long constrain(long x, long a, long b) {
    if (x < a) return a;
    if (x > b) return b;
    return x;
}

// 로봇 손 제어
void update_robot_hand_linear(void)
{
    for (int i = 0; i < 5; i++)
    {
        long adc_val = adc_values[i];

        long pulse = map(adc_val, ADC_FLAT, ADC_BENT, SERVO_MIN, SERVO_MAX);
        pulse = constrain(pulse, SERVO_MIN, SERVO_MAX);

        switch (i)
        {
            case 0: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, pulse); break;
            case 1: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, pulse); break;
            case 2: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, pulse); break;
            case 3: __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, pulse); break;
            case 4: __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse); break;
        }
    }
}
```

플렉스 센서 ADC 값을 굽힘 범위 (ADC_FLAT ~ ADC_BENT)에서 서보 펄스 범위 (SERVO_MIN ~ SERVO_MAX)로 변환한다. 변환된 값을 최소~최대 범위 안으로 제한한다. 각 손가락 (0~4)에 해당하는 PWM 채널에 그 펄스 값을 넣어 서보 모터를 움직인다.

실패 사례 분석 및 해결

본 프로젝트 진행 중 발생한 주요 기술적 난관과 이를 해결하기 위한 엔지니어링 접근 방식은 다음과 같다. 특히 로봇 핸드의 기구적 설계 및 재질 특성으로 인한 문제가 두드러졌다.

문제 1: 필라멘트 강성으로 인한 구동 실패

현상: 3D 프린팅된 로봇 손가락 관절이 필라멘트 소재(PLA)의 높은 강성으로 인해 뻣뻣하여, 서보모터의 토크만으로는 굽힘 동작이 원활하지 않거나 모터에 과부하가 걸리는 현상이 발생함

원인: 적층형 3D 프린팅 방식의 특성상 관절 부위의 유연성이 부족함.

해결: 인두기를 사용하여 관절 부위의 필라멘트를 가열하여 부드럽게 만듦

한계: 열처리를 통해 관절을 얇고 부드럽게 만들었으나, 동시에 재료의 두께와 내구성을 저하, 관절 부위가'끊어지기 직전의 매우 얇은 상태로 유지되어, 반복적인 굽힘-펴짐 동작 시 끊어질 위험이 극도로 높아짐. 결과적으로 장기적인 신뢰성을 확보할 수 없어 최종적인 시스템 구현 실패로 결론지음.

문제 2: 실전 동작 복원력 부족

현상: 낚싯줄을 당겨 손가락을 굽히는 동작은 정상적으로 수행되었으나, 줄을 풀었을 때 손가락이 다시 펴지는 동작이 이루어지지 않음.

원인:

단동식 제어: 서보모터가 당기는 힘(굽힘)만 제공하고 미는 힘(펴짐)은 제공하지 못하는 와이어 구동 방식의 한계.

복원력 부재: 손가락 관절 자체에 탄성이 없거나, 이를 보조할 스프링/고무줄 등의 탄성체가 설계에 반영되지 않아 원래 위치로 돌아오지 못함.

분석: 로봇 손 설계 시, 굽힘 동작뿐만 아니라 **수동적 복원력** 또는 **이중 와이어** 구조에 대한 고려가 필수적임을 확인함.

Result (결과): 재질 유연화 작업을 통해 굽힘 동작은 성공시켰으나, 펴짐 동작을 위한 탄성 복원 메커니즘의 부재로 인해 완전한 양방향 제어에는 실패함. 이는 향후

탄성 소재(TPU) 활용이나 스프링 내장형 설계 도입의 필요성을 확인함.

IV. CONCLUSION

본 프로젝트는 기반 손동작 인식 시스템과 IMU 센서, 블루투스 통신(TTS 포함)을 결합하여 수어 기반 입력을 한글 문자 및 음성으로 변환하는 시스템을 구현하였다. 전체 동작 과정은 손가락 굽힘 패턴을 자모로 매핑하고, 엄지를 사용하여 자음·모음을 구분하며, 완성된 글자는 손목 회전 제스처(180°)를 통해 전송하는 구조로 구성되었다.

실험 결과, 각 손가락의 굽힘 정도는 ADC 를 통해 안정적으로 자모 패턴으로 변환되었으며, 생성된 자모는 UTF-8 인코딩을 통해 올바른 한글 글자로 조합되었고, 블루투스(HC-06) 통신을 통해 PC 로 전송된 후 TTS 엔진(pyttsx3)에 의해 음성 출력까지 정상적으로 수행되었다.

본 프로젝트를 통해 센서 신호 처리–임베디드 제어–문자 조합 알고리즘–블루투스 통신–PC 음성출력에 이르는 전체 시스템 설계 과정을 경험할 수 있었고, 제스처 기반 입력 장치 설계의 실현 가능성을 검증하였다. 향후 보완점으로는 플렉스 센서 보정 자동화, 더 넓은 자모 세트 지원, 단어 단위 자동완성 확장 및 TTS 품질 개선 등을 고려할 수 있다.

REFERENCES

[1] 임종수, *STM32 Cortex-M 완벽활용서 Vol.1*, 버무림, 2024.